

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Microprocessors Microcontrollers

Lab 1 report

LED Animations

Advisor(s): Phan Van Sy

Student(s): Nguyen Thanh Luan 2452711

HO CHI MINH CITY, SEPTEMBER 2026



Contents

1	Exercise 1	4
2	Exercise 2	5
3	Exercise 3	7
4	Exercise 4	9
5	Exercise 5	12
6	Exercise 6	15
7	Exercise 7	17
8	Exercise 8	17
9	Exercise 9	18
10	Exercise 10	19

List of Figures

1.1	<i>State transitions for 2 LEDs</i>	4
1.2	Proteus Schematic for Exercise 1. The source code and project files can be accessed at: GitHub Repository	4
2.1	Proteus Schematic for Exercise 2. The source code and project files can be accessed at: GitHub Repository	6
3.1	<i>Reference design for a 4 way traffic light</i>	7
3.2	Proteus Schematic for Exercise 3. The source code and project files can be accessed at: GitHub Repository	8
4.1	<i>Display a number on 7 segment LED</i>	10
4.2	Proteus Schematic for Exercise 4. The source code and project files can be accessed at: GitHub Repository	11
5.1	Proteus Schematic for Exercise 5. The source code and project files can be accessed at: GitHub Repository	13
6.1	<i>12 LEDs for an analog clock</i>	15



6.2	Proteus Schematic for Exercise 6. The source code and project files can be accessed at: GitHub Repository	16
10.1	Proteus Schematic for Exercise 10. The source code and project files can be accessed at: GitHub Repository	19

List of Tables

Listings

1 Exercise 1

From the simulation on Proteus, one more LED is connected to pin **PA6** of the STM32 (negative pin of the LED is connected to PA6). The component suggested in this exercise is **LED-YELLOW**, which can be found from the device list.

In this exercise, the status of two LEDs are switched every 2 seconds, as demonstrated in the figure bellow.

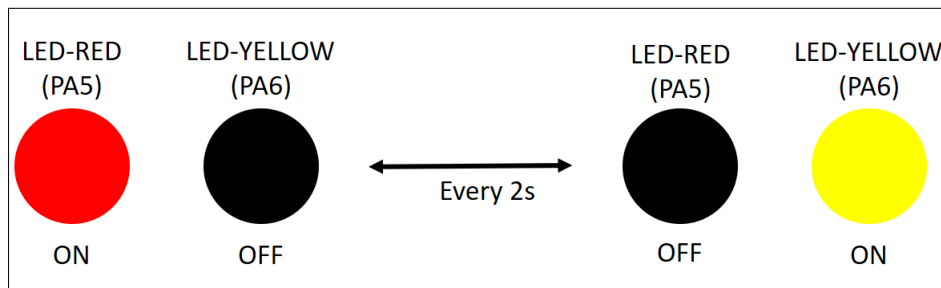


Figure 1.1: *State transitions for 2 LEDs*

Report 1

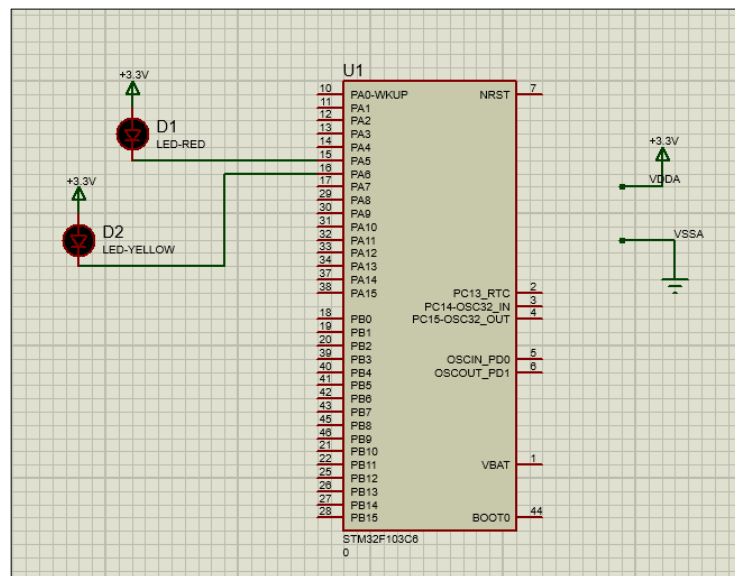


Figure 1.2: Proteus Schematic for Exercise 1. The source code and project files can be accessed at: [GitHub Repository](#)

Report 2



```
1  while (1)
2  {
3      // Toggle the states of both RED and YELLOW LEDs simultaneously
4      HAL_GPIO_TogglePin(GPIOA, LED_RED_Pin);
5      HAL_GPIO_TogglePin(GPIOA, LED_YELLOW_Pin);
6
7      // Create a delay of 2 seconds (2000 ms)
8      HAL_Delay(2000);
9
10     /* USER CODE END WHILE */
11
12     /* USER CODE BEGIN 3 */
13 }
14 /* USER CODE END 3 */
```

2 Exercise 2

Extend the first exercise to simulate the behavior of a traffic light. A third LED, named **LED-GREEN** is added to the system, which is connected to **PA7**. A cycle in this traffic light is 5 seconds for the RED, 2 seconds for the YELLOW and 3 seconds for the GREEN. The LED-GREEN is also controlled by its negative pin.

Similarly, the report in this exercise includes the schematic of your circuit and a your source code in the while loop.

Report 1:

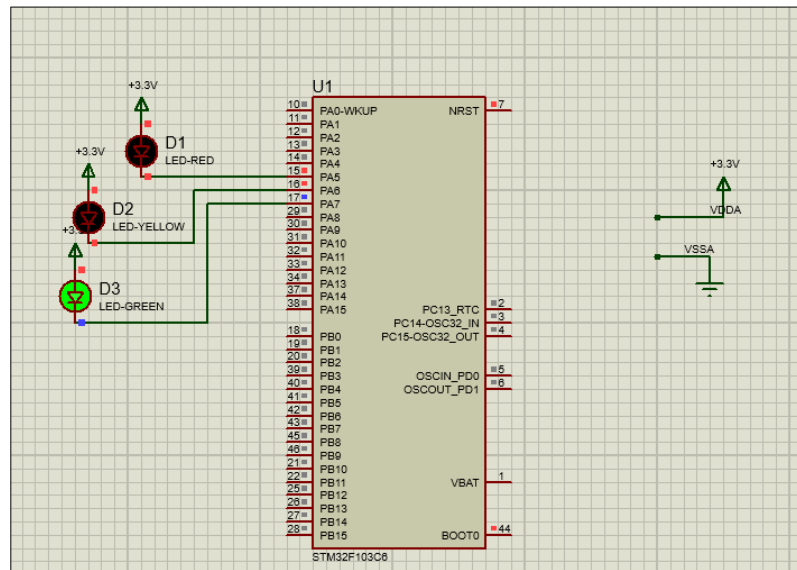


Figure 2.1: Proteus Schematic for Exercise 2. The source code and project files can be accessed at: [GitHub Repository](#)

Report 2:

```
1  /* Infinite loop */
2  /* USER CODE BEGIN WHILE */
3  while (1)
4  {
5      HAL_GPIO_WritePin(GPIOA, LED_RED_Pin, GPIO_PIN_RESET);
6      HAL_GPIO_WritePin(GPIOA, LED_YELLOW_Pin, GPIO_PIN_SET);
7      HAL_GPIO_WritePin(GPIOA, LED_GREEN_Pin, GPIO_PIN_SET);
8      HAL_Delay(5000);
9
10     HAL_GPIO_WritePin(GPIOA, LED_RED_Pin, GPIO_PIN_SET);
11     HAL_GPIO_WritePin(GPIOA, LED_YELLOW_Pin, GPIO_PIN_SET);
12     HAL_GPIO_WritePin(GPIOA, LED_GREEN_Pin, GPIO_PIN_RESET);
13     HAL_Delay(3000);
14
15     HAL_GPIO_WritePin(GPIOA, LED_RED_Pin, GPIO_PIN_SET);
16     HAL_GPIO_WritePin(GPIOA, LED_YELLOW_Pin, GPIO_PIN_RESET);
17     HAL_GPIO_WritePin(GPIOA, LED_GREEN_Pin, GPIO_PIN_SET);
18     HAL_Delay(2000);
```

```

19  /* USER CODE END WHILE */
20
21  /* USER CODE BEGIN 3 */
22  }
23  /* USER CODE END 3 */

```

3 Exercise 3

Extend to the 4-way traffic light. Arrange 12 LEDs in a nice shape to simulate the behaviors of a traffic light. A reference design can be found in the figure bellow.

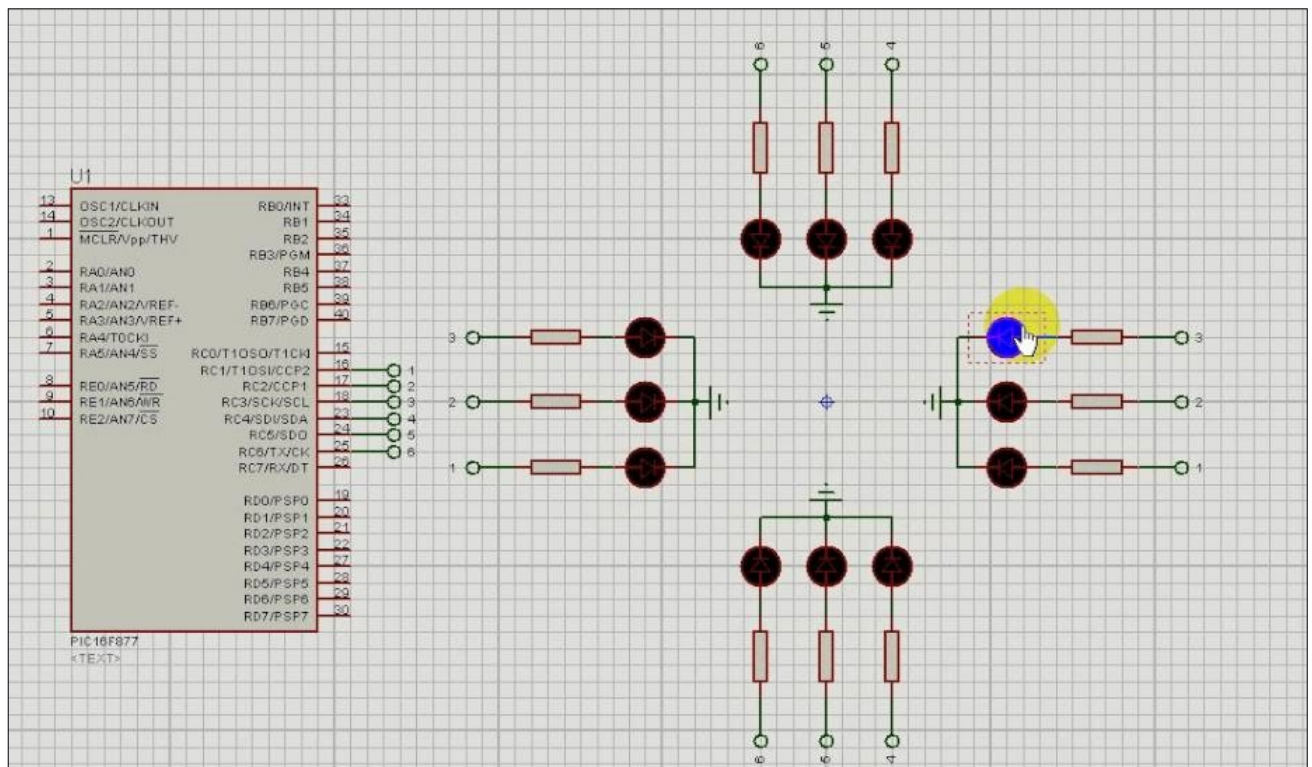


Figure 3.1: Reference design for a 4 way traffic light

Report 1:

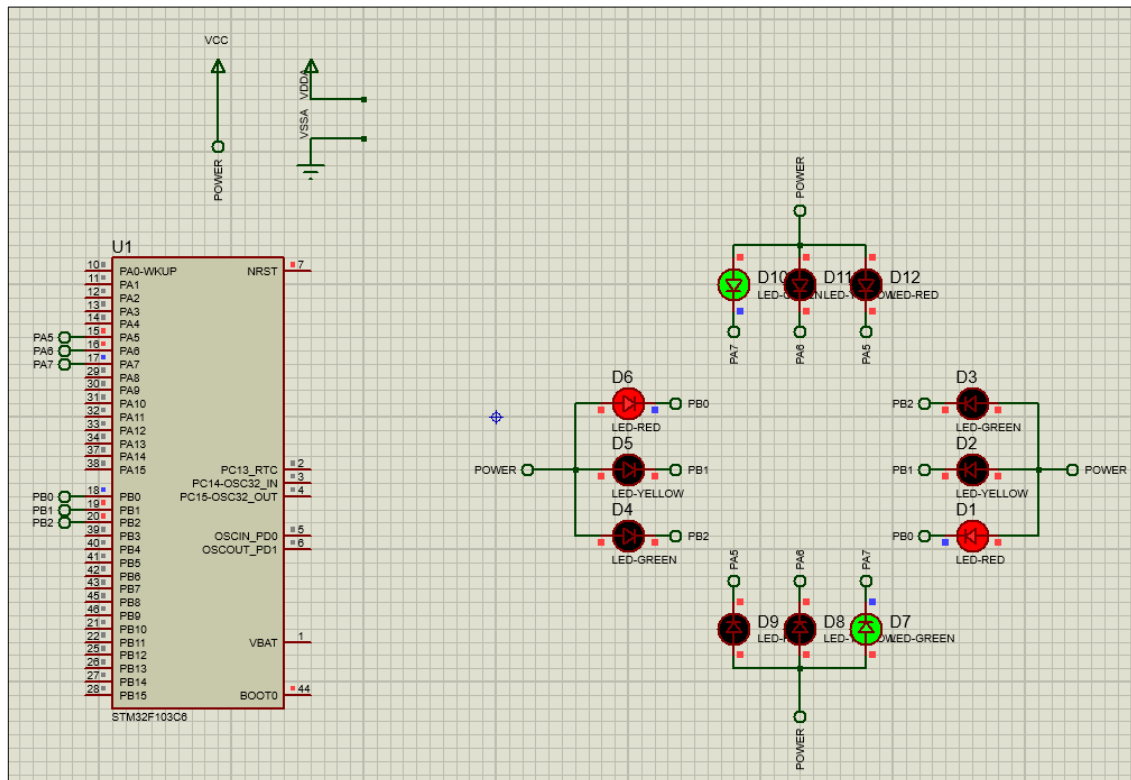


Figure 3.2: Proteus Schematic for Exercise 3. The source code and project files can be accessed at: [GitHub Repository](#)

Report 2:

```

1  /* Infinite loop */
2  /* USER CODE BEGIN WHILE */
3  while (1)
4  {
5      //GREEN-RED (3s)
6      HAL_GPIO_WritePin(GPIOA, LED_RED_1_Pin | LED_YELLOW_1_Pin,
7                          GPIO_PIN_SET);
8      HAL_GPIO_WritePin(GPIOA, LED_GREEN_1_Pin, GPIO_PIN_RESET);
9      HAL_GPIO_WritePin(GPIOB, LED_YELLOW_2_Pin | LED_GREEN_2_Pin,
10                         GPIO_PIN_SET);
11     HAL_GPIO_WritePin(GPIOB, LED_RED_2_Pin, GPIO_PIN_RESET);
12     HAL_Delay(3000);

    // YELLOW - RED (2s)

```



```
13 HAL_GPIO_WritePin(GPIOA, LED_GREEN_1_Pin, GPIO_PIN_SET);
14 HAL_GPIO_WritePin(GPIOA, LED_YELLOW_1_Pin, GPIO_PIN_RESET);
15 HAL_Delay(2000);
16
17 // RED - GREEN (3s)
18 HAL_GPIO_WritePin(GPIOA, LED_YELLOW_1_Pin, GPIO_PIN_SET);
19 HAL_GPIO_WritePin(GPIOA, LED_RED_1_Pin, GPIO_PIN_RESET);
20 HAL_GPIO_WritePin(GPIOB, LED_RED_2_Pin, GPIO_PIN_SET);
21 HAL_GPIO_WritePin(GPIOB, LED_GREEN_2_Pin, GPIO_PIN_RESET);
22 HAL_Delay(3000);
23
24 // RED - YELLOW (2s)
25 HAL_GPIO_WritePin(GPIOB, LED_GREEN_2_Pin, GPIO_PIN_SET);
26 HAL_GPIO_WritePin(GPIOB, LED_YELLOW_2_Pin, GPIO_PIN_RESET);
27
28 HAL_Delay(2000);
29 /* USER CODE END WHILE */
30 /* USER CODE BEGIN 3 */
31 }
32 /* USER CODE END 3 */
```

4 Exercise 4

Add **only one 7 led segment** to the schematic in Exercise 3. This component can be found in Proteus by the keyword **7SEG-COM-ANODE**. For this device, the common pin should be connected to the power supply and other pins are supposed to be connected to PB0 to PB6. Therefore, to turn-on a segment in this 7SEG, the STM32 pin should be in logic 0 (0V).

Implement a function named **display7SEG(int num)**. The input for this function is from 0 to 9 and the outputs are listed as following:

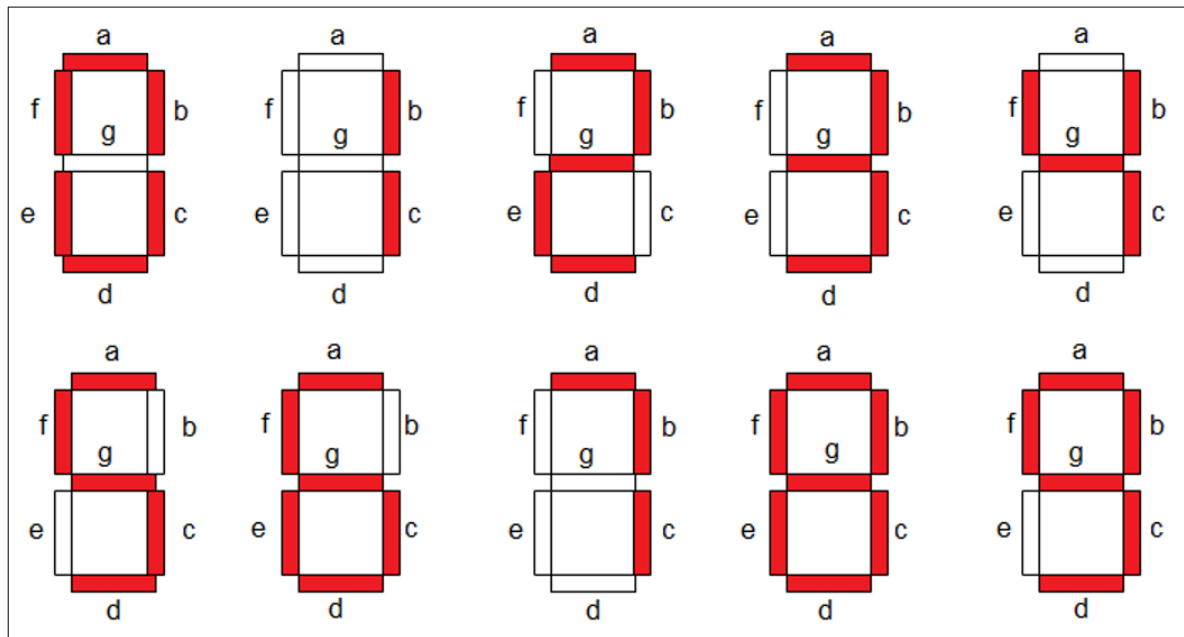


Figure 4.1: *Display a number on 7 segment LED*

This function is invoked in the while loop for testing as following:

```
1  int counter = 0;
2  while (1){
3      if(counter >= 10) counter = 0;
4      display7SEG(counter++);
5      HAL_Delay(1000);
6
7  }
```

Report 1:

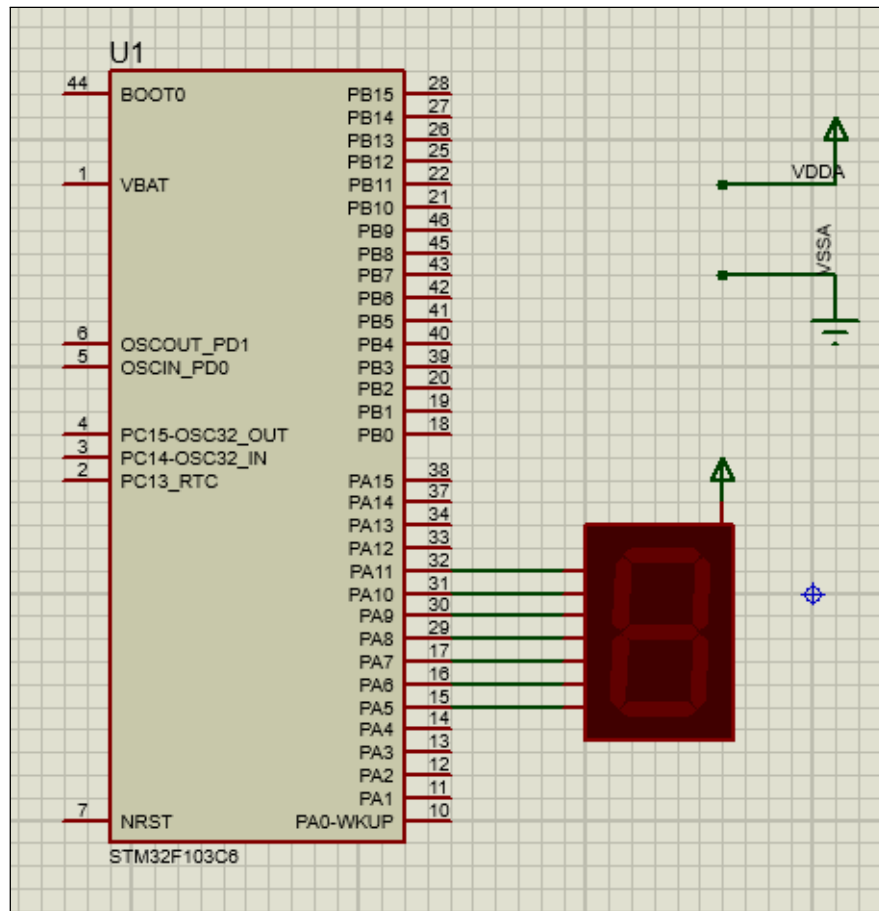


Figure 4.2: Proteus Schematic for Exercise 4. The source code and project files can be accessed at: [GitHub Repository](#)

Report 2:

```
1  /* USER CODE BEGIN 4 */
2  void display7SEG(int num) {
3      uint8_t seg_data[10] = {0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82,
4          0xF8, 0x80, 0x90};
5
6      if (num < 0 || num > 9) return;
7
8      uint8_t data = seg_data[num];
9
10     //a
11     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, (data >> 0) & 1);
```

```
11 //b
12 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, (data >> 1) & 1);
13 //c
14 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, (data >> 2) & 1);
15 //d
16 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, (data >> 3) & 1);
17 //e
18 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, (data >> 4) & 1);
19 //f
20 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, (data >> 5) & 1);
21 //g
22 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, (data >> 6) & 1);
23 }
24 /* USER CODE END 4 */
```

5 Exercise 5

Integrate the 7SEG-LED to the 4 way traffic light. In this case, the 7SEG-LED is used to display countdown value.

In this exercise, only source code is required to present. The function display7SEG in previous exercise can be re-used.

Report 1:

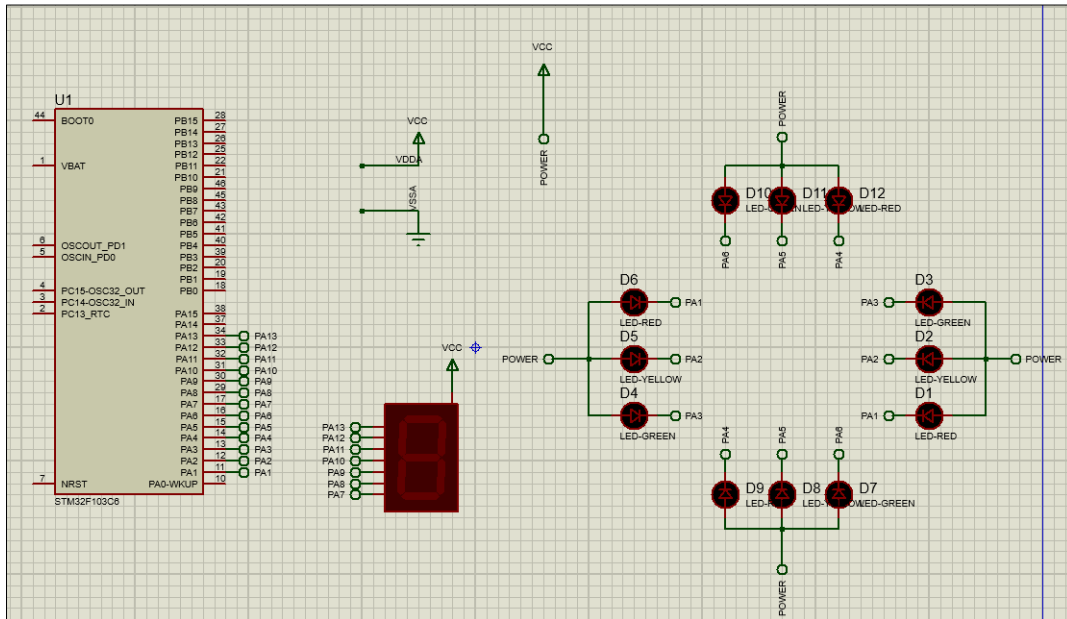


Figure 5.1: Proteus Schematic for Exercise 5. The source code and project files can be accessed at: [GitHub Repository](#)

Report 2:

```

1  /* Infinite loop */
2  /* USER CODE BEGIN WHILE */
3  while (1)
4  {
5      // GREEN - RED (3s)
6      HAL_GPIO_WritePin(GPIOA, LED_RED_1_Pin | LED_YELLOW_1_Pin,
7                          GPIO_PIN_SET);
8      HAL_GPIO_WritePin(GPIOA, LED_GREEN_1_Pin, GPIO_PIN_RESET);
9      HAL_GPIO_WritePin(GPIOA, LED_YELLOW_2_Pin | LED_GREEN_2_Pin,
10                         GPIO_PIN_SET);
11     HAL_GPIO_WritePin(GPIOA, LED_RED_2_Pin, GPIO_PIN_RESET);
12
13     for(int i = 3; i > 0; i--) {
14         display7SEG(i);
15         HAL_Delay(1000);
16     }
17 }

```



```
16 // YELLOW - RED (2s)
17 HAL_GPIO_WritePin(GPIOA, LED_GREEN_1_Pin, GPIO_PIN_SET);
18 HAL_GPIO_WritePin(GPIOA, LED_YELLOW_1_Pin, GPIO_PIN_RESET);
19
20 for(int i = 2; i > 0; i--) {
21     display7SEG(i);
22     HAL_Delay(1000);
23 }
24
25 // RED - GREEN (3s)
26 HAL_GPIO_WritePin(GPIOA, LED_YELLOW_1_Pin, GPIO_PIN_SET);
27 HAL_GPIO_WritePin(GPIOA, LED_RED_1_Pin, GPIO_PIN_RESET);
28 HAL_GPIO_WritePin(GPIOA, LED_RED_2_Pin, GPIO_PIN_SET);
29 HAL_GPIO_WritePin(GPIOA, LED_GREEN_2_Pin, GPIO_PIN_RESET);
30
31 for(int i = 3; i > 0; i--) {
32     display7SEG(i);
33     HAL_Delay(1000);
34 }
35
36 // RED - YELLOW (2s)
37 HAL_GPIO_WritePin(GPIOA, LED_GREEN_2_Pin, GPIO_PIN_SET);
38 HAL_GPIO_WritePin(GPIOA, LED_YELLOW_2_Pin, GPIO_PIN_RESET);
39
40 for(int i = 2; i > 0; i--) {
41     display7SEG(i);
42     HAL_Delay(1000);
43 }
44 /* USER CODE END WHILE */
45
46 /* USER CODE BEGIN 3 */
47 }
48 /* USER CODE END 3 */
```

6 Exercise 6

In this exercise, a new Proteus schematic is designed to simulate an analog clock, with 12 different number. The connections for 12 LEDs are supposed from PA4 to PA15 of the STM32. The arrangement of 12 LEDs is depicted as follows.

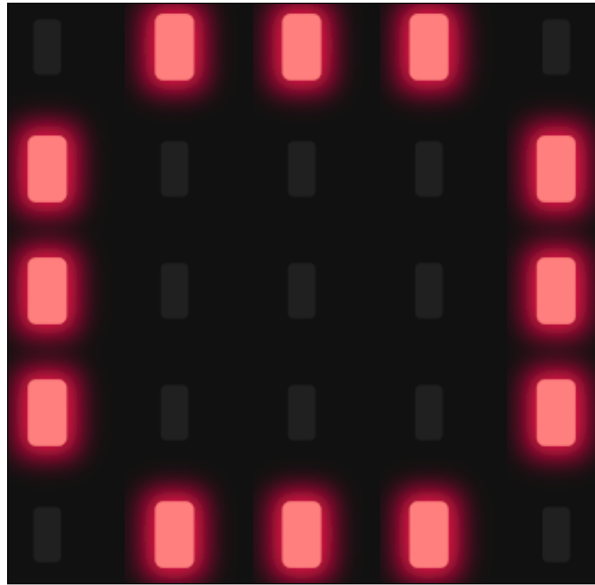


Figure 6.1: *12 LEDs for an analog clock*

Report 1:

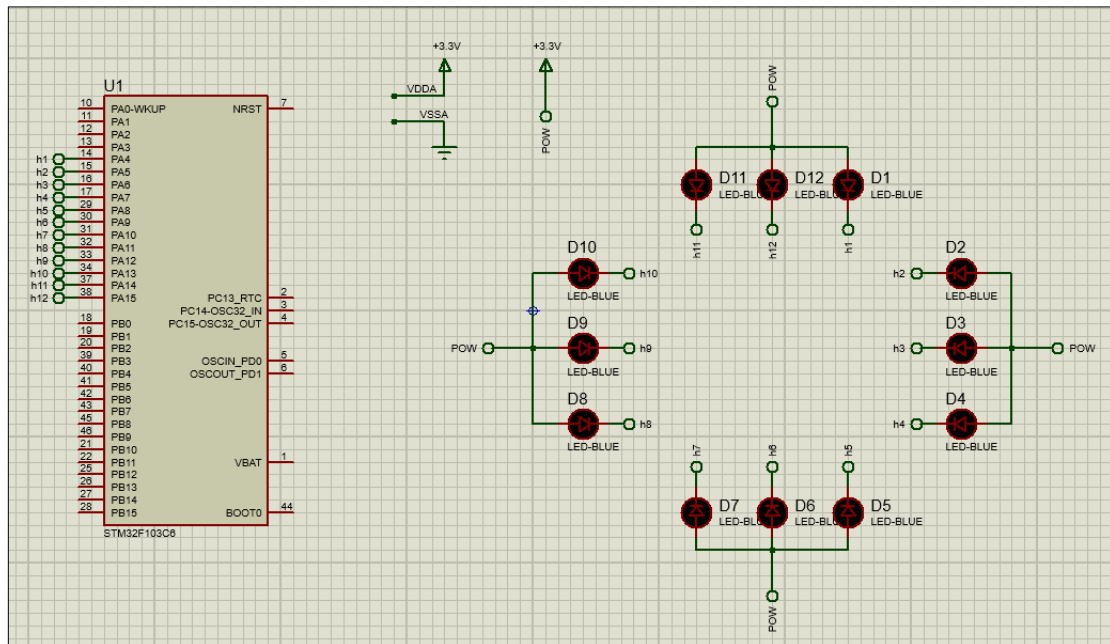


Figure 6.2: Proteus Schematic for Exercise 6. The source code and project files can be accessed at: [GitHub Repository](#)

Report 2:

```
1  /* Infinite loop */
2  /* USER CODE BEGIN WHILE */
3  while (1)
4  {
5      for (int i = 0; i < 12; i++) {
6          HAL_GPIO_WritePin(GPIOA, clock_leds[i], GPIO_PIN_RESET);
7          HAL_Delay(200);
8          HAL_GPIO_WritePin(GPIOA, clock_leds[i], GPIO_PIN_SET);
9      }
10     /* USER CODE END WHILE */
11     /* USER CODE BEGIN 3 */
12 }
13 /* USER CODE END 3 */
```



7 Exercise 7

Implement a function named `clearAllClock()` to turn off all 12 LEDs. Present the source code of this function.

```
1 void clearAllClock(){
2     //TODO
3     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4 | GPIO_PIN_5 |
4         GPIO_PIN_6 | GPIO_PIN_7 | GPIO_PIN_8 |
5         GPIO_PIN_9 | GPIO_PIN_10 | GPIO_PIN_11 |
6         GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14 |
7         GPIO_PIN_15, GPIO_PIN_SET);
8 }
```

8 Exercise 8

Implement a function named `setNumberOnClock(int num)`. The input for this function is from **0** to **11** and an appropriate LED is turn on. Present the source code of this function.

```
1 void setNumberOnClock(int num) {
2     uint16_t target_pin;
3
4     switch(num) {
5         case 0: target_pin = GPIO_PIN_15; break;
6         case 1: target_pin = GPIO_PIN_4; break;
7         case 2: target_pin = GPIO_PIN_5; break;
8         case 3: target_pin = GPIO_PIN_6; break;
9         case 4: target_pin = GPIO_PIN_7; break;
10        case 5: target_pin = GPIO_PIN_8; break;
11        case 6: target_pin = GPIO_PIN_9; break;
12        case 7: target_pin = GPIO_PIN_10; break;
13        case 8: target_pin = GPIO_PIN_11; break;
14        case 9: target_pin = GPIO_PIN_12; break;
15        case 10: target_pin = GPIO_PIN_13; break;
16        case 11: target_pin = GPIO_PIN_14; break;
```



```
17     default: return;
18 }
19
20 HAL_GPIO_WritePin(GPIOA, target_pin, GPIO_PIN_RESET);
21 }
```

9 Exercise 9

Implement a function named **clearNumberOnClock(int num)**. The input for this function is from **0 to 11** and an appropriate LED is turn off.

```
1 void clearNumberOnClock(int num) {
2     uint16_t target_pin;
3
4     switch(num) {
5         case 0: target_pin = GPIO_PIN_15; break;
6         case 1: target_pin = GPIO_PIN_4; break;
7         case 2: target_pin = GPIO_PIN_5; break;
8         case 3: target_pin = GPIO_PIN_6; break;
9         case 4: target_pin = GPIO_PIN_7; break;
10        case 5: target_pin = GPIO_PIN_8; break;
11        case 6: target_pin = GPIO_PIN_9; break;
12        case 7: target_pin = GPIO_PIN_10; break;
13        case 8: target_pin = GPIO_PIN_11; break;
14        case 9: target_pin = GPIO_PIN_12; break;
15        case 10: target_pin = GPIO_PIN_13; break;
16        case 11: target_pin = GPIO_PIN_14; break;
17        default: return;
18    }
19
20    HAL_GPIO_WritePin(GPIOA, target_pin, GPIO_PIN_SET);
21 }
```

10 Exercise 10

Integrate the whole system and use 12 LEDs to display a clock. At a given time, there are only 3 LEDs are turn on for hour, minute and second information.

Report 1:

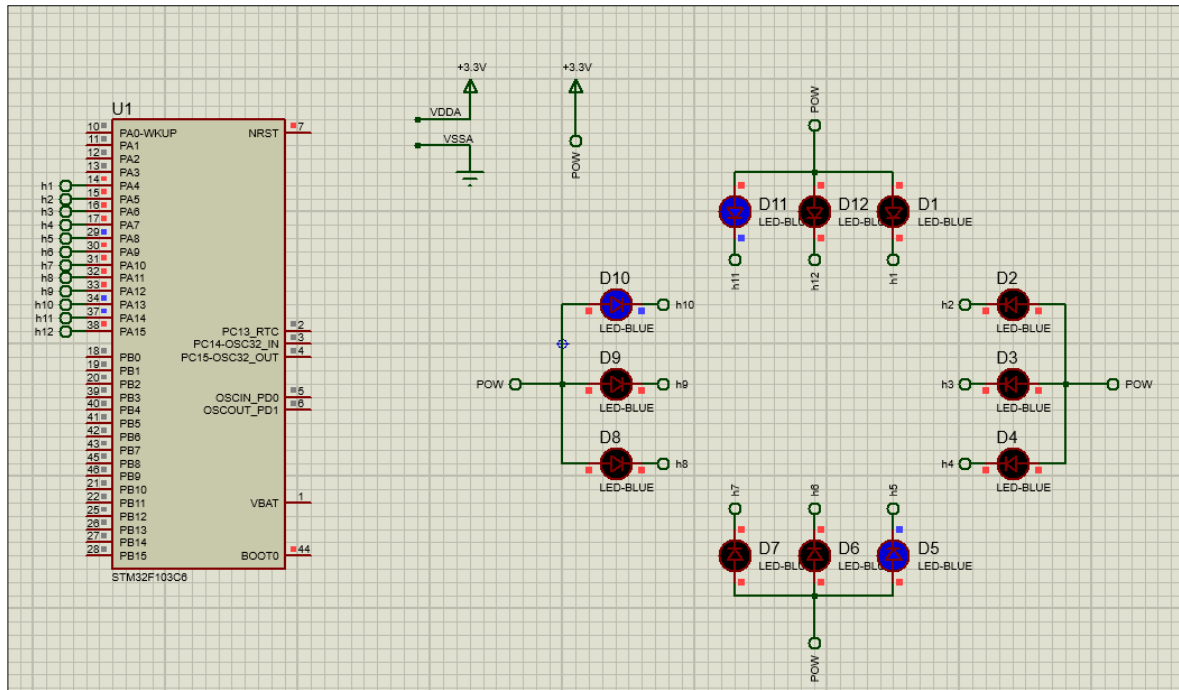


Figure 10.1: Proteus Schematic for Exercise 10. The source code and project files can be accessed at: [GitHub Repository](#)

Report 2:

```
1  /* Infinite loop */
2  /* USER CODE BEGIN WHILE */
3  while (1)
4  {
5      clearAllClock();
6
7      setNumberOnClock(hour);
8      setNumberOnClock(minute / 5);
9      setNumberOnClock(second / 5);
10
11     second++;
```



```
12
13     if (second >= 60) {
14         second = 0;
15         minute++;
16     }
17
18     if (minute >= 60) {
19         minute = 0;
20         hour++;
21     }
22
23     if (hour >= 12) {
24         hour = 0;
25     }
26
27     HAL_Delay(30);
28     /* USER CODE END WHILE */
29     /* USER CODE BEGIN 3 */
30 }
31 /* USER CODE END 3 */
```



References

- [1] Paul Adrien Maurice Dirac. *The Principles of Quantum Mechanics*. International series of monographs on physics. Clarendon Press, 1981.
- [2] Albert Einstein. Zur Elektrodynamik bewegter Körper. (German) [On the electrodynamics of moving bodies]. *Annalen der Physik*, 322(10):891–921, 1905.
- [3] Michel Goossens, Frank Mittelbach, and Alexander Samarin. *The L^AT_EX Companion*. Addison-Wesley, Reading, Massachusetts, 1993.
- [4] George D. Greenwade. The Comprehensive Tex Archive Network (CTAN). *TUGBoat*, 14(3):342–351, 1993.
- [5] Gareth Halfacree. The official bbc micro:bit user guide.
- [6] Donald Knuth. Knuth: Computers and typesetting.
- [7] Donald E. Knuth. *The Art of Computer Programming*. Four volumes. Addison-Wesley, 1968. Seven volumes planned.
- [8] Donald E. Knuth. *Fundamental Algorithms*, chapter 1.2. Addison-Wesley, 1973.